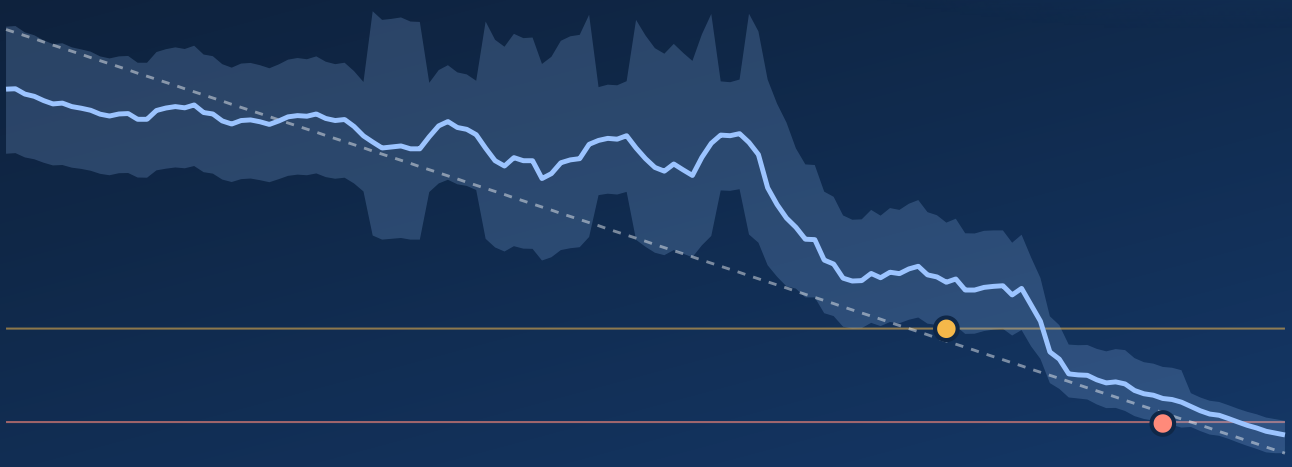
AI FOR RESILIENT AND SUSTAINABLE SUPPLY CHAINS · ROUND 2 SUBMISSION

Predict Wear Before the Next Take-off

AI-based estimation of gradual aircraft engine wear, turned into a spare-parts plan before the next departure.

Team BuriBuriZaimon



● Plan alert: 36 flights before end of life ● Review alert: 13 flights before end of life

Remaining useful life of held-out engine FD001-R-074 after every flight: estimate (blue line), 80% band, true life (dashed)

ONE PASS AFTER EVERY LANDING

1. **Landing** — A flight completes; new sensor readings for both engines
2. **Estimate wear** — RUL with a calibrated 80% band and top sensors
3. **Plan the part** — Need-by time; reserve, transfer or order, ranked
4. **Alert** — The dashboard flags what needs action before departure
5. **Decide** — An authorised planner approves or overrides

13.6 cycles
RUL error (RMSE) on 100 unseen NASA test engines. Baseline: 18.4

85%
of true values inside the calibrated 80% band on held-out engines

695 → 3 h
hours aircraft wait on the ground for parts, reactive vs predictive*

2,131 → 193 kg
CO2e from freighting parts in the same scenario*

INSIDE THIS SUBMISSION

1 · Problem & Solution | 2 · Architecture | 3 · Technical Approach | 4 · Prototype | 5 · Feasibility & Impact | 6 · Future Scope

*Synthetic supply-chain scenario replaying 20 engines to the end of their simulated life; it shows the mechanism, not real-world savings. Engine data: NASA C-MAPSS FD001 (simulated). The system is decision support only: authorised maintenance personnel make every decision.

SUMMARY

At a glance

After every landing, our prototype estimates how much useful life each engine has left, with a calibrated uncertainty band. It then turns that estimate into a spare-parts plan that an authorised planner approves before the next departure.

What we built

- **An LSTM sequence model** that reads the last 30 flights of 14 engine sensors and estimates remaining useful life (RUL), with a calibrated 80% band and the sensors behind each estimate.
- **A planning engine** that turns the cautious lower edge of the band into a need-by time and ranks every way to get the part there: reserve, transfer or order, by on-time arrival, cost and CO2e.
- **A fleet optimizer** (mixed-integer programme) that shares scarce spares so none is promised twice.
- **A working dashboard** with post-landing alerts, approve-or-override and a decision log, plus a replay that measures the effect against waiting for failure.

What we measured

Measure	Result
RUL error on 100 unseen test engines (RMSE)	**13.6 cycles** baseline 18.4
True values inside the 80% band, held-out engines	**85%** raw MC dropout 70%
Review alert before simulated end of life, all 20 held-out engines	≥ 12 flights
Unplanned removals in the replay*	20 → 0
Hours waiting on the ground for parts*	694.8 → 3.2
Freight emissions*	2,131 → 193 kg CO2e
Life left unused at removal (the trade-off)*	0 → 9.9 cycles

*Synthetic supply-chain scenario. Engine data is simulated (NASA C-MAPSS FD001).

Contents

1 **Problem & Solution** 3
Why late visibility of wear hurts the supply chain, and our post-landing answer

2 **Architecture** 4
System design, components and the workflow after every landing

3 **Technical Approach** 5
Datasets, models, uncertainty, explainability and decision methods

4 **Prototype** 8
The working dashboard, what it does and how to demonstrate it

5 **Feasibility & Impact** 10
Practical feasibility, beneficiaries, measured impact and scalability

6 **Future Scope** 12
Validation, extensions and deployment, plus references

Responsible use. The system recommends; it never reserves, moves or orders a part on its own. Qualified maintenance personnel and approved procedures remain responsible for inspections, maintenance actions and airworthiness decisions.

1 PROBLEM & SOLUTION

Spot wear early, have the part ready

Engines wear a little on every flight. Sensors record it, but the readings rarely become an early estimate that maintenance and supply-chain teams can act on before the next departure.

The problem

Components degrade gradually over many flights. When wear is recognised late, the right part may not be at the right station, and teams fall back on urgent sourcing: expedited orders, air freight and, at worst, an aircraft held on the ground (AOG) until a part arrives. Late visibility shortens every downstream lead time, rules out cheaper and lower-emission transport, and leaves stock in the wrong places. Earlier work has studied how imperfect remaining-useful-life (RUL) predictions can drive maintenance scheduling [3]; we connect such predictions to spare-parts readiness.

Core problem. Maintenance and supply-chain teams need an actionable estimate of wear after landing and before the next departure, so the right component can be inspected, prepared or made available in time.

Who it affects

Stakeholder	Today	What they need
Airlines and fleet operations	Little advance visibility of component wear and maintenance needs	A post-landing wear summary and a forecast before the next departure
MRO teams (maintenance, repair and overhaul)	Work is hard to plan when wear trends are unclear	Health indicators, trend history and human-review alerts
Spare-parts and inventory teams	Uncertain demand leads to urgent orders or stock in the wrong place	Early demand signals, stock checks and replenishment recommendations
Suppliers and logistics providers	Short notice limits sourcing and transport choices	Earlier demand visibility and time to choose feasible delivery options

Our solution

A decision-support system that runs one pass after every landing and hands a ranked parts plan to a person:

1. **Landing** — A flight completes; new sensor readings for both engines
2. **Estimate wear** — RUL with a calibrated 80% band and top sensors
3. **Plan the part** — Need-by time; reserve, transfer or order, ranked
4. **Alert** — The dashboard flags what needs action before departure
5. **Decide** — An authorised planner approves or overrides

What makes it different

Gradual wear, not in-flight failure
Reads trends across completed flights and reviews them on the ground, before the next departure.

Plans on honest uncertainty
Acts on the cautious lower edge of a calibrated band and measures how often the band holds.

Wear connected to parts
Estimates flow into stock, transfer and supplier decisions, shared across the fleet.

Sustainability measured, not assumed
Every option carries kg CO_2e from cited factors [4]; the policy's effect is measured in a replay.

2 ARCHITECTURE

One pass from sensor history to a reviewed parts plan

Every landing triggers five steps. Data flows up from the stores beneath, and a person's approval flows back into the stock that the next plan sees.

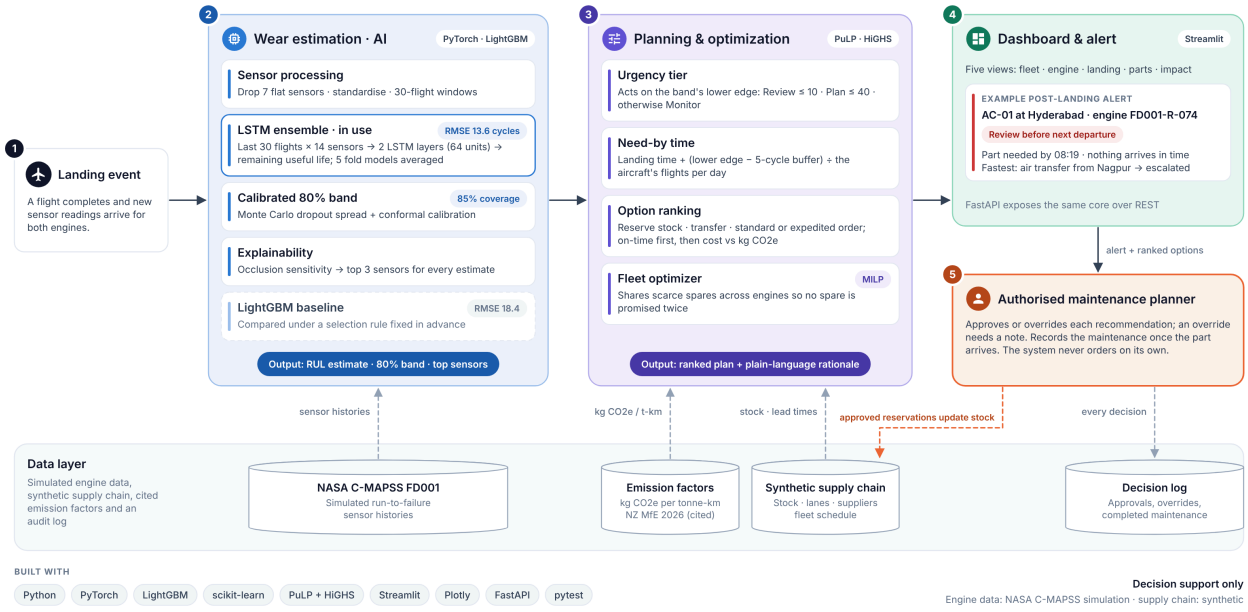


Figure 1. System architecture. The numbered steps run after every landing. Dashed lines show data reads, the decision log, and the feedback loop through which approved reservations update stock.

Components

Layer	What it does	Built with
Wear estimation	Builds 30-flight sensor windows; an LSTM ensemble estimates RUL; a Monte Carlo dropout band is calibrated by conformal prediction; occlusion names the top 3 sensors	PyTorch; LightGBM baseline for comparison
Model-to-planner contract	One row per engine per flight: estimate, band, top sensors, model version	CSV (`predictions.csv`)
Planning engine	Urgency tier and need-by time from the lower edge; builds and ranks reserve, transfer, standard and expedited options	Python
Fleet optimizer	Allocates scarce spares across engines in one mixed-integer programme	PuLP with the HiGHS solver
Dashboard and API	Five views, post-landing alerts, approve or override, decision log; the same core over REST	Streamlit, Plotly, FastAPI
Data layer	NASA C-MAPSS FD001; synthetic network, stock, suppliers and fleet; cited emission factors; decision log	CSV files

Design choices

- **Plan on the lower edge.** Overstating remaining life is the unsafe direction.
- **Evaluate honestly.** 20 engines are never used for training, early stopping or calibration.
- **Pick the model by a rule written first:** band coverage 70–90%, then lowest test RMSE.
- **Keep deployment light.** The dashboard reads precomputed predictions, so the cloud app needs no ML libraries.
- **Keep people in charge.** Approvals reserve stock and are logged; a person records completed maintenance.
- **Close the loop.** Reserved stock is removed from what later plans can use.

3 TECHNICAL APPROACH

Data, models and AI methods

A 5-fold LSTM ensemble estimates remaining useful life with an RMSE of 13.6 cycles on the 100 official NASA test engines, against 18.4 for a LightGBM baseline built and tested the same way.

Datasets

Dataset	What it holds	How we use it
NASA C-MAPSS, subset FD001 [1, 2]	100 training engines run to failure (20,631 cycles); 100 test engines stopped before failure (13,096 cycles) plus their true RUL; 21 sensors; one operating condition; one fault mode (high-pressure compressor degradation)	Engine wear. One cycle is treated as one flight
Synthetic supply chain (generated, fixed seed)	7 locations, 5 parts, 10 suppliers, stock, transfer lanes, 10 aircraft with 20 engines	Parts planning; clearly labelled synthetic
Freight emission factors [4]	kg CO2e per tonne-km for road, air (with radiative forcing) and container shipping	Emissions of each delivery option

Preparation. Seven flat sensors are dropped, leaving 14. RUL labels are capped at 125 cycles, the standard target for C-MAPSS. Data is split by engine, never by row: 80 engines train the models in 5 engine-wise folds, 20 are held out completely, and the 100 official test engines are scored once, at their last cycle.

Models and AI methods

	LightGBM baseline	LSTM ensemble (in use)
Input	84 features: each sensor's value, 5- and 10-cycle mean and spread, 10-cycle slope	The last 30 cycles of 14 standardised sensors
Model	Gradient-boosted trees	2 LSTM layers × 64 units → dense 32; variational dropout 0.25; Adam, early stopping; 5 fold models averaged
80% band	10th–90th percentile of cross-validated errors in each predicted-RUL range	Monte Carlo dropout spread (25 fixed masks per model), calibrated by conformal prediction on out-of-fold errors
Explanation	TreeSHAP (exact for trees)	Occlusion: each sensor's window is set to its average and the change in the estimate is measured
Training time	About 30 seconds	About 6 minutes on 2 CPU cores

Results

Measure	LightGBM	LSTM
Test RMSE, 100 official test engines (true RUL capped at 125)	18.4 cycles	**13.6 cycles**
NASA PHM08 score, which penalises late estimates (lower is better)	1,298	**291**
RMSE on the 20 held-out engines, every cycle	17.7 cycles	**14.2 cycles**
80% band coverage on held-out engines (target 80%)	82%	85%
80% band coverage on test engines	86%	88%
Mean band width, held-out engines	46 cycles	**38 cycles**
Plan alert: median flights left when it fires	62	52
Review alert: median flights left (worst engine)	17 (12)	16 (12)

The LSTM is more accurate, and its band is narrower yet still covers more of the truth. Under both models every held-out engine was flagged for review at least 12 flights before its simulated end of life.

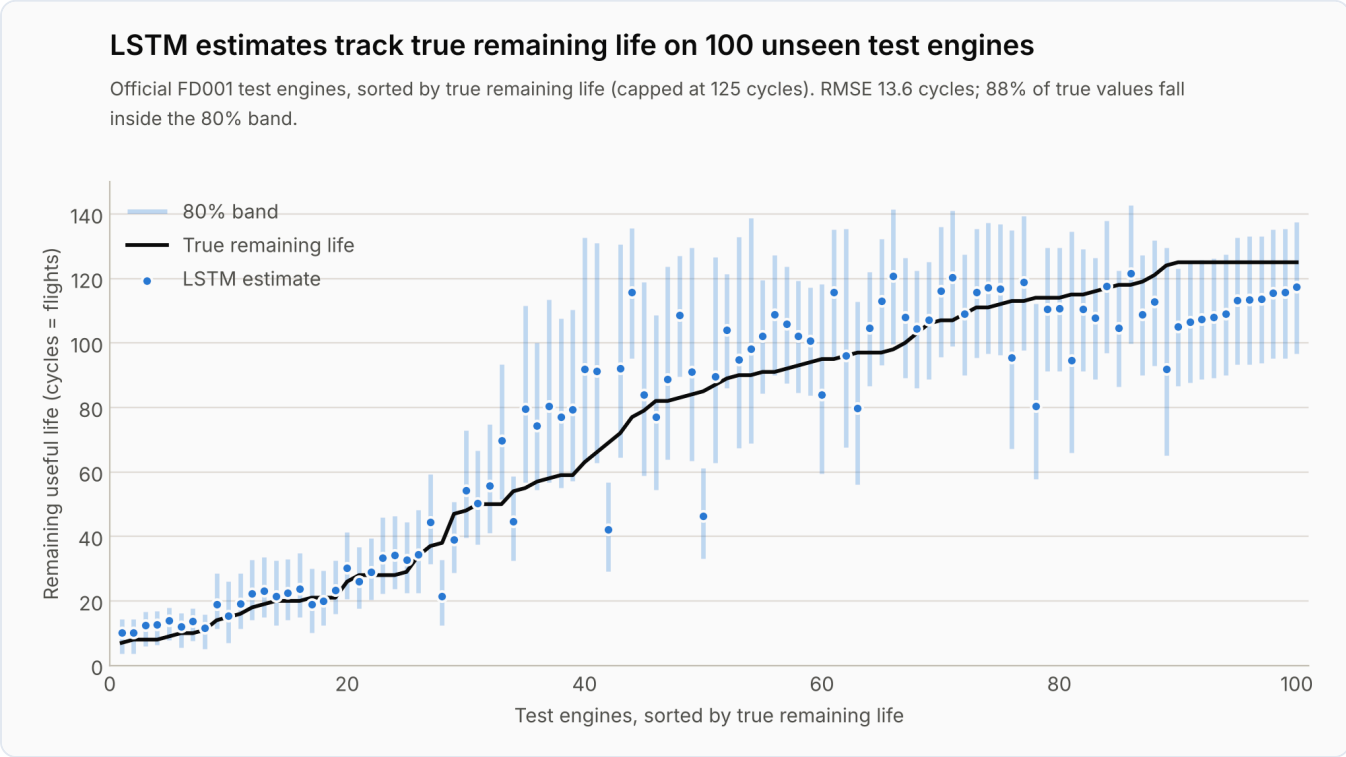


Figure 2. LSTM estimates (dots) and 80% bands (bars) against true remaining life on the 100 official test engines, sorted by true life.

Uncertainty you can trust

Read as a normal distribution, the raw Monte Carlo dropout spread covers only 70% of true values on held-out engines, and just 44%–49% between 30 and 100 cycles. Conformal calibration fixes this. Within each predicted-RUL range, the out-of-fold errors, scaled by their own spread, set how far the band reaches. Fixed dropout masks make every estimate repeatable: the inference function reproduces the dashboard's numbers exactly.

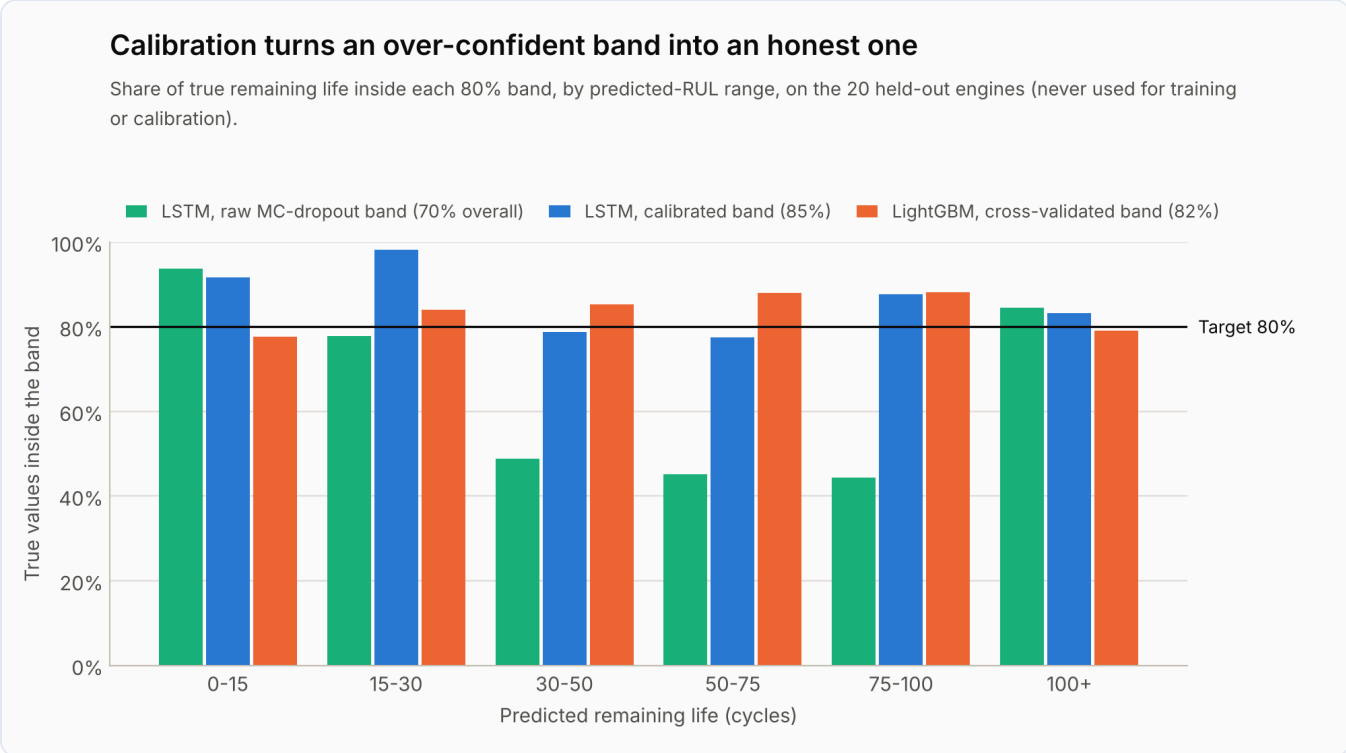


Figure 3. Share of true values inside each 80% band, by predicted-RUL range, on the 20 held-out engines.

Explainability

Every estimate names its three most influential sensors. Across the held-out engines, both models rank core speed (Nc or its corrected form NRc), LPT outlet temperature (T50) and HPC static pressure (Ps30) among their top five sensors. These are signals of the compressor degradation simulated in FD001.

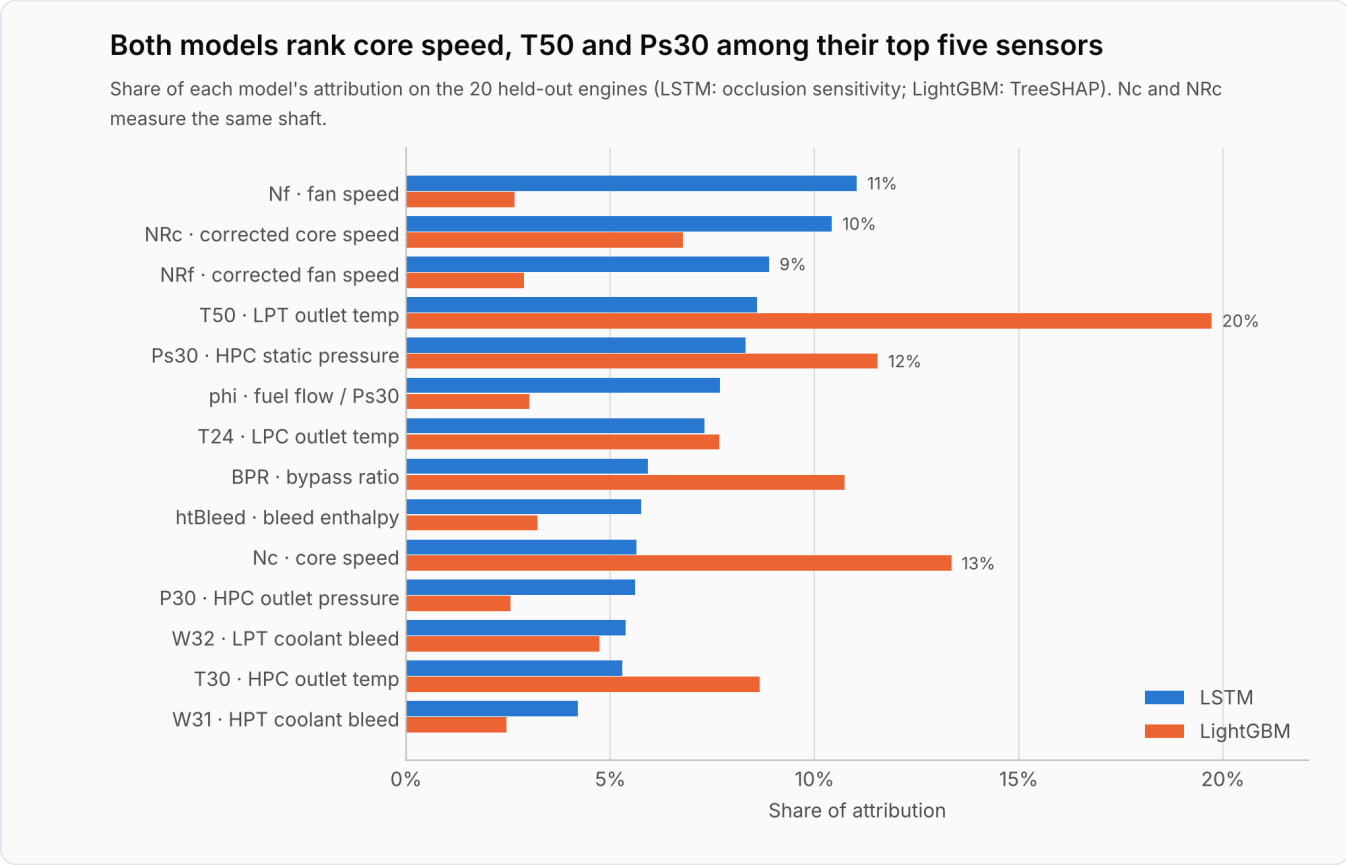


Figure 4. Share of each model's attribution on the held-out engines (LSTM: occlusion sensitivity; LightGBM: TreeSHAP).

From estimate to decision

Step	Method
Urgency tier	On the band's lower edge: review before next departure at ≤ 10 cycles, plan maintenance at ≤ 40, otherwise monitor (illustrative thresholds, not airworthiness limits)
Need-by time	Landing time + (lower edge − 5-cycle buffer) ÷ the aircraft's flights per day
Option ranking	Reserve stock, transfer by road or air, standard or expedited order. On-time options are scored by normalised cost and kg CO2e with a user-set weight; if nothing arrives in time, the fastest option is escalated
Emissions	Part weight × distance × kg CO2e per tonne-km for the transport mode [4]
Fleet optimizer	Mixed-integer programme: one option per engine, stock limits per location, late options heavily penalised (PuLP + HiGHS)
Policy replay	Discrete-event replay of the fleet to end of life under reactive and predictive policies with shared stock

Technologies

Python · pandas · NumPy · scikit-learn · PyTorch (LSTM) · LightGBM · PuLP + HiGHS · Streamlit · Plotly · FastAPI · matplotlib · pytest · GitHub · Streamlit Community Cloud

4 PROTOTYPE

A working end-to-end prototype

The Streamlit dashboard runs the full loop on a synthetic fleet of 10 aircraft and 20 engines: simulate a landing, see the updated estimate, get a ranked parts plan, approve it, and measure the effect.

Implemented functionality

View	What works
Fleet overview	Engines ranked by urgency with estimate, lower edge, tier, need-by time, next departure and part status; spare stock by location
Engine detail	RUL history with the 80% band and thresholds; the true life line (simulation only); top sensors and sensor trend charts
Simulate landing	Land one aircraft or the next in the fleet; post-landing review card with tier changes and time to next departure; an engine at end of life grounds its aircraft
Parts plan	Recommendation with a plain-language rationale; every option with arrival, on-time check, cost and kg CO2e; approve, override with a note, cancel, record maintenance; fleet plan from the optimizer; decision log
Impact	Reactive vs predictive replay with the unused-life trade-off; model quality and the LSTM vs LightGBM comparison

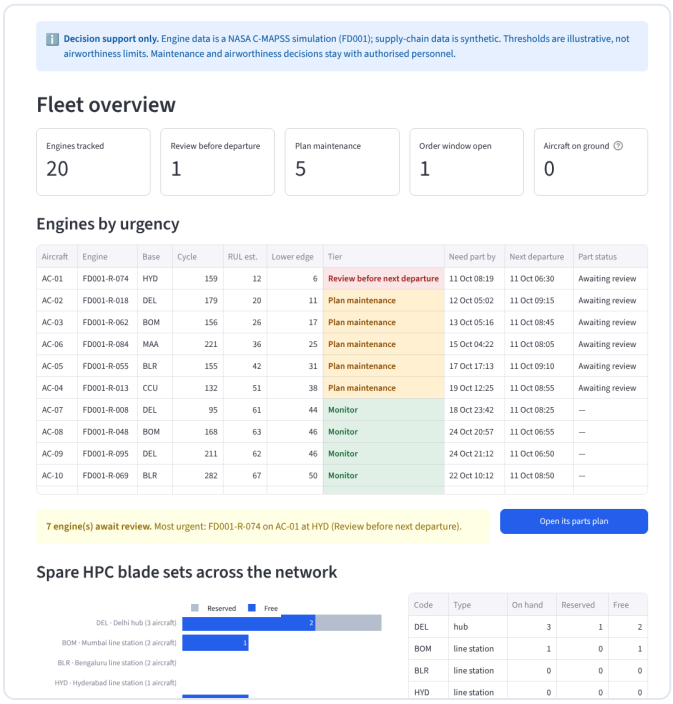


Figure 5. Fleet overview: engines ranked by urgency.

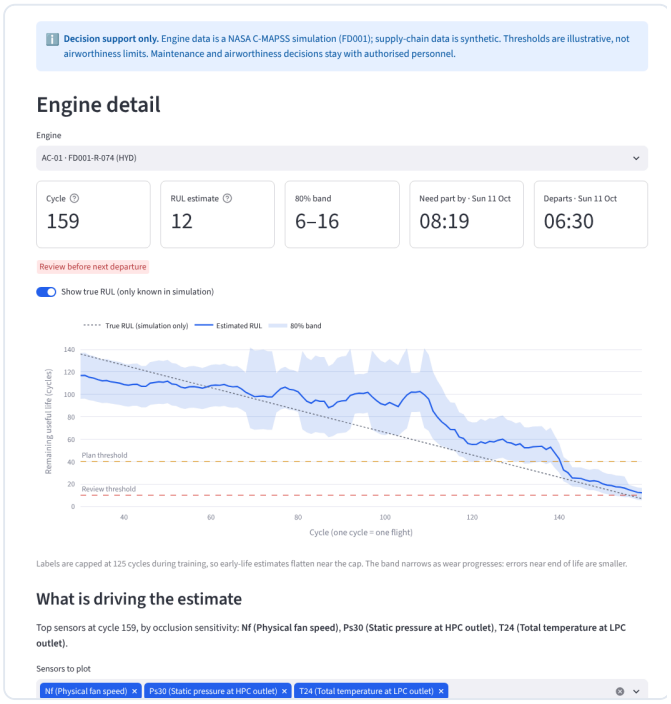


Figure 6. Engine detail: estimate, band, thresholds and top sensors.

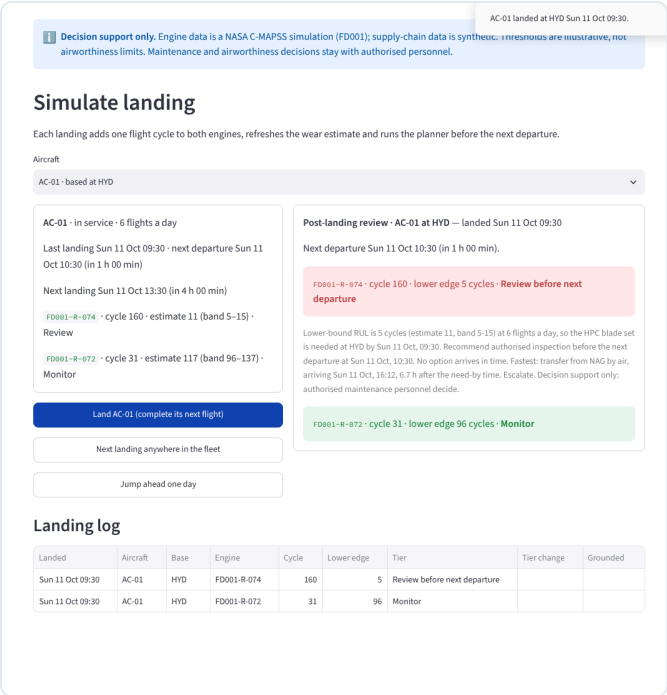


Figure 7. Post-landing review after AC-01 lands.

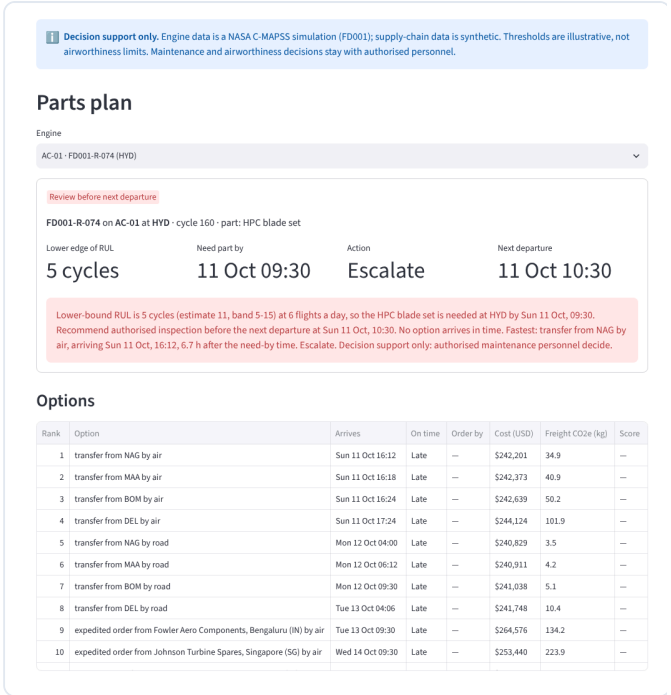


Figure 8. Parts plan: recommendation, rationale and every option.

Demonstration path

1. **Fleet overview.** One engine (AC-01 at Hyderabad) needs review before its 06:30 departure, and only 6 free spare HPC blade sets exist across the network.
2. **Engine detail.** AC-01's engine: the band narrows towards the review threshold; the true life line confirms it.
3. **Simulate landing.** AC-01 lands; the review card shows the lower edge, the need-by time and the time to departure.
4. **Parts plan.** No option reaches Hyderabad in time, so the fastest (air transfer from Nagpur) is escalated with its shortfall. Approve it.
5. **Fleet plan.** Planned one by one, the engines would claim more spares than exist; the optimizer reassigns them in one click.
6. **Cost vs emissions.** For AC-08 at Mumbai, cost-only picks a sea order from Singapore; emissions-only reserves the spare already in Mumbai.
7. **Impact.** Reactive vs predictive on the same fleet, the unused-life trade-off, and the two models compared.

Engineering quality

- **32 automated tests** cover models, inference, planning rules, the optimizer, simulations, the API and every dashboard view.
- **Reproducible:** fixed seeds; the inference function reproduces the dashboard's estimates exactly.
- **Deployable:** the dashboard needs six runtime packages and runs on Streamlit Community Cloud; a FastAPI layer exposes the same core over REST.

5 FEASIBILITY & IMPACT

Feasible today, with measured impact

The prototype runs on public data and free tools, and fits the window between landing and the next departure. In a synthetic replay of 20 engines it removed unplanned removals and almost all waiting for parts.

Feasibility

Technical

Public NASA data and open-source tools. The LSTM trains in about 6 minutes on 2 CPU cores. Planning all 20 engines takes about 0.1 s and the fleet optimizer under 0.05 s. The dashboard runs without ML libraries and deploys free.

Operational

It works after landing, when the aircraft is on the ground. On the demo engine, the plan alert came 36 flights and the review alert 13 flights before simulated end of life: about 6 and 2 days at 6 flights a day.

Human oversight

Every recommendation is reviewed by an authorised planner, overrides need a note, and every decision is logged. Nothing is ordered automatically.

What real use would need

Authorised engine and maintenance data, real stock and lead times, thresholds set by qualified experts, and validation before any operational trial.

Beneficiaries

Who	How they benefit
Airlines and fleet operations	Earlier warning of engines that need attention, and fewer aircraft waiting on the ground for parts
MRO teams	Health indicators with honest uncertainty, the sensors behind each estimate, and time to plan the work
Spare-parts and inventory teams	Ranked, on-time options and a fleet-wide allocation that never promises a spare twice
Suppliers and logistics providers	Demand signals early enough to use standard lead times and slower, lower-emission transport

Measured impact (synthetic scenario)

Measure	Reactive (wait for failure)	Predictive (this planner)
Unscheduled removals	20	**0**
Hours aircraft wait on the ground for parts (AOG)	694.8	**3.2**
Air shipments	17	**1**
Expedited orders	14	**0**
Parts cost incl. part value and logistics (USD)	$5,154,887	**$4,901,334**
Freight emissions	2,131 kg CO2e	**193 kg CO2e**
Mean life left unused at removal	**0 cycles**	9.9 cycles

Same fleet, stock and suppliers under both policies; all 20 engines replayed to the end of their simulated life. The supply chain is synthetic, so this shows the mechanism, not real-world savings. The remaining 3.2 hours come from one engine already in the review tier when the replay starts, with no spare able to reach its base in time.

The trade-off. Removing engines before they fail leaves life unused: about 10 cycles per engine here. The safety buffer and thresholds control this balance and would be set by qualified experts.

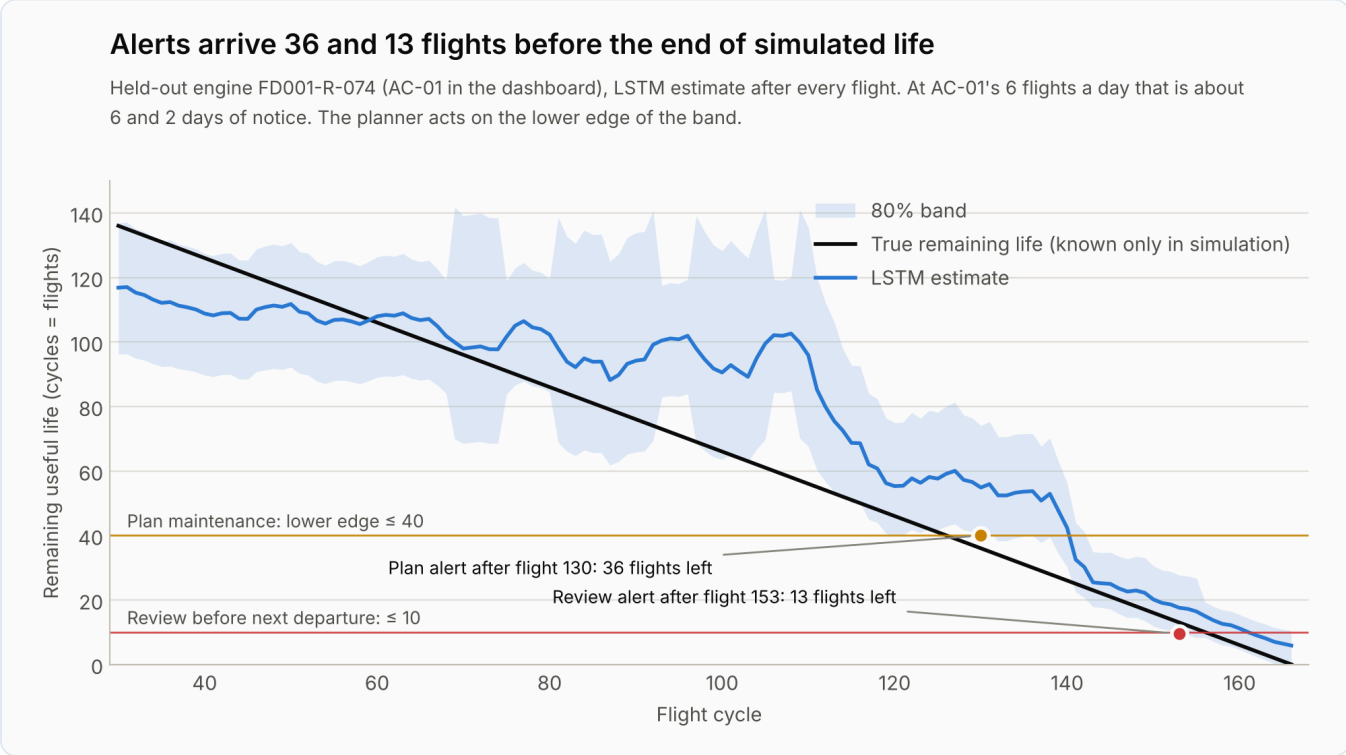


Figure 9. Alert lead time on the demo engine. The planner acts on the lower edge of the band, so alerts arrive while there is still time to prepare the part.

Scalability

- **More engines:** planning and allocation take well under a second for the current fleet, and the optimizer has one decision per engine and option.
- **More components and conditions:** the model-to-planner contract is one table, so new models (for example for FD002–FD004) plug in unchanged.
- **Real systems:** the REST API exposes the same core, ready to connect to maintenance and ERP systems for live stock and lead times.

6 FUTURE SCOPE

From prototype to pilot

The next step is validation on authorised airline data with an industry partner. The architecture already separates the parts that would change: the model, the data stores and the integrations.

1 · Validate

- Test on real engine and maintenance records with a partner
- Recalibrate the band on real data
- Have qualified experts set thresholds and the safety buffer
- Run in shadow mode alongside current planning

2 · Extend the AI

- FD002–FD004: six operating conditions and two fault modes
- Map each fault mode to the parts it needs
- Monitor drift and retrain as fleets age
- Coverage guarantees per RUL range and per engine type

3 · Integrate and deploy

- Connect maintenance and ERP systems for live stock and lead times
- Multi-location inventory and reorder policies
- Supplier reliability and live carrier emissions
- Edge processing of sensor data at the aircraft or gate

Longer-term possibilities

- **Digital twins** of engines that combine the wear estimate with maintenance history.
- **Network-wide planning** that positions spares across stations before demand appears.
- **Sustainability reporting** that records the kg CO2e of every parts decision.

References

1. NASA. C-MAPSS Jet Engine Simulated Data. NASA Open Data Portal. https://data.nasa.gov/dataset/cmapss-jet-engine-simulated-data
2. A. Saxena, K. Goebel, D. Simon, N. Eklund. Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. International Conference on Prognostics and Health Management, 2008.
3. I. de Pater, A. Reijns, M. Mitici. Alarm-based predictive maintenance scheduling for aircraft engines with imperfect Remaining Useful Life prognostics. Reliability Engineering & System Safety, 2022. https://doi.org/10.1016/j.ress.2022.108341
4. New Zealand Ministry for the Environment. Measuring Emissions Catalogue 2026: Freight. https://measuringemissionsguide.environment.govt.nz/8_freight.html

Data notice and AI usage declaration

Engine data is the NASA C-MAPSS simulation (FD001). All supply-chain data (locations, stock, prices, lead times, suppliers and fleet schedule) is synthetic and labelled as such. Generative AI (Claude, by Anthropic) assisted with code, tests, documentation and the drafting of this document. Every result reported here was produced by running the project's code and was reviewed by the team.